

응용 · CODEX CLI · VS CODE 강의 교안

Coding Agent 실전

AI기반 TDD 마스터 과정

개념에서 코드까지, 추적성을 잃지 않는 개발 (Concept-to-Code)

4계층 검증 구조

계약 ID

추적성 매트릭스

Dual-Track TDD $\equiv$ ARRR

하네스 엔지니어링

하루 8시간 집중 과정 · 08:30-17:30 | 강의 40% + 실습 60% | 개발자 및 입문자 대상

2026 · 김대경 Ph.D.

과정 개요 — 하루 만에 C2C 개발 흐름을 체득한다

항목	내용
교육 대상	개발자 및 입문자 (Git 기초 숙지 전제)
교육 시간	8시간 — 08:30~17:30 (점심 11:45~13:00) · 0장(사전 과제)은 시간에서 제외
교육 방식	강의 40% + 실습 60% (개념 → 시연 → 개별 실습)
도구 스택	VS Code · Codex CLI(터미널 AI 에이전트) · Codex IDE 확장(VS Code)
사전 과제	0장(설치·CLI 기본기)은 교육 전 개별 완료 — 당일엔 체크리스트만 확인
관통 예제	장바구니 할인 계산기 — 계약 ID 6개(INV·E·AC·U)를 ARRR로 완성

핵심 목표 — 개념이 코드까지 추적되는(C2C) 개발 흐름을 1일 안에 체득한다. 산출물: docs/PRD.md · tests/ · src/ · docs/traceability.md

00

환경 준비 — 설치·CLI 기본기는 미리 끝낸다

Chapter 0 · 사전 과제 (강의 시간에서 제외). 0장은 교육 전 개별 완료하고, 당일 0교시에는 체크리스트만 5분 확인한 뒤 바로 이론으로 진입한다.

준비물 점검 — 시작 전에 갖춰야 할 것

개인 PC

Windows · macOS · Linux (듀얼 모니터 권장)

GitHub · ChatGPT 계정

무료 등급으로 CLI·Code Assist 로그인

Node.js 20+ · Python 3.11+ · Git

CLI 설치와 pytest·Flask 실습에 사용

환경 — 버전 확인

```
# 이 네 줄이 통과해야 시작
$ node --version # v20.x 이상
$ python --version # 3.11.x 이상
$ git --version # 2.x
$ codex --version # 없으면 0.2에서 설치
```

⚠ 가장 흔한 사전 준비 실패 원인 — Node.js가 20 미만이면 Codex CLI 설치가 실패한다. 반드시 먼저 확인한다.

도구 스택과 설치 — 세 가지의 역할 분담

VS Code

에디터 + 통합 터미널

코드 열람·수동 편집, 터미널에서 `codex.pytest` 실행

Codex CLI

터미널 AI 에이전트

다중 파일 편집·테스트 자동 실행·자율 반복 (ARRR 주력)

Codex IDE 확장(VS Code)

VS Code 확장(채팅·Agent)

코드 질의·인라인 편집·Agent 탭 (CLI와 동일 엔진)

Codex CLI 설치 명령

```
# codex-cli 설치  
npm install -g @openai/codex
```

💡 수업 시 Codex IDE 확장(VS Code)는 사용하지 않는다.
CLI 중심으로 진행한다.

Codex CLI 기본기 — 대화 시작 방법 5가지

대화 시작 방법	무엇·언제	예시 (codex-at-practice-python)
codex	여러 단계의 대화형 개발	codex
codex exec "..."	한 번 분석하고 종료	codex exec "@src/utils/math_utils.py 설명해줘"
@경로	코드/문서를 명시적 컨텍스트로 지정	@src/utils/math_utils.py 경계조건을 찾아줘
!명령	실제 프로젝트 명령 실행	!python -m src.main, !pytest -q
/명령	Codex CLI 기능 실행	/help, /model, /mcp

승인 모드 — plan(읽기 전용) · default(매번 승인, 권장 시작점) · auto_edit(편집만 자동) · yolo(전부 자동, 격리 저장소에서만).
△ 처음엔 반드시 default로 시작한다.

01

Dual-Track TDD — RED → GREEN 으로 추적 가능한 개발

1부 · 왜 테스트를 먼저 두는가. Codex CLI · Python 3.12 · pytest · Git | 하루 8시간 집중 과정

미니 체험 ① 문제의식 — 바이브 코딩과 C2C 단절

바이브 코딩 (Vibe Coding)

AI에게 즉흥적 요청을 반복해 기능을 얻어내는 방식. 동작은 하지만 “이 코드가 어느 요구·개념에서 나왔는가”를 역추적할 수 없다 = C2C 단절.

- 요구 만족의 근거가 없다
- 무엇을 왜 고쳤는지 나중에 추적 불가
- 에이전트가 오래 일할수록 “통과한 듯 보이는” 코드가 는다

탄생과 확산 (2025년 초)

용어의 등장:

- 바이브 코딩 (Vibe Coding)의 역사는 2025년 2월 오픈AI의 공동 창립자 안드레이 카파시 (Andrej Karpathy)가 이 용어를 처음 사용하면서 시작
- 패러다임 변화: 개발자가 직접 코드를 짜는 '구현 (How)' 중심에서, AI에게 방향성을 지시하는 '의도 (What)' 중심의 개발로 패러다임으로

발전과 한계 (2025년 말 ~ 2026년)

- 생성된 코드를 인간이 온전히 이해하지 못해 실제 운영 환경에서 보안 결함과 유지보수 문제
- 새로운 논쟁: AI 시대에 인간 중심 언어가 필요한가에 대한 논의와 기계 친화적 언어 제안 등



미니 체험

미니 체험 ② — 같은 기능을 TDD로 재구현

RED — 의도된 실패

프롬프트: “tests/test_calc.py 에 test_add 테스트만.
src/calc.py 는 만들지 마.”

“add가 없으니 pytest는 의도된 실패(RED).”

GREEN — 최소 통과

프롬프트: “방금 테스트를 통과시키도록 add(a,b)를
가장 단순하게 구현해줘.”

“1 passed — 이 테스트는 이제 안전자산.”

REFACTOR — 안전한 개선

프롬프트: “불필요한 주석·가독성 저해 요소를 정리하
고 pytest 통과를 확인해줘.”

“여전히 1 passed — 불안하지 않다.”

미니 체험 ③ — ARRR 예제 — 사칙계산기

장바구니(4장)에 들어가기 전, 더 작은 예제로 ARRR 네 박자를 끝까지 — 여기서만 Repeat(새 요구사항으로 다음 사이클)까지 봅니다.

A = Ask (RED)

테스트 3개를 먼저 — 빈 문자열→0 ·
단일 숫자·콤마 두 숫자. 구현이 없으
니 ImportError/실패.

R = Respond (GREEN)

통과시키는 가장 단순한 구현만. 과
도한 추상화·미래 대비 코드 금지.

R = Refine (REFACTOR)

동작은 그대로, 책임만 분리 —
_is_blank(검증) / _calculate_sum(계
산).

R = Repeat

새 요구: 세미콜론 구분자. 다시 Ask(
실패 테스트)부터 → _split_numbers
추출.

핵심 원칙 — RED→GREEN→REFACTOR 사이클을 절대 건너뛰지 않는다. 실패를 먼저 확인한 뒤에만 구현한다. “최소한의 코드”만 쓴다. 리팩터링은 외부 동작을 바꾸지 않는 선에서만.

왜 “테스트 먼저”인가

판단의 조급성 · Discovery

문서는 바람(needs)의 나열이었을 뿐, 서로가 합의하고 증거로 판정할 수 있는 가설·수용조건이 아니다.

[시스템 '개발' 현장의 모습]



행동의 조급성 · Delivery

검증 근거 없이 구현이 앞서면, 통과한 것처럼 보이지만 실제로는 아닌 코드가 쌓인다.

[시스템 '개발' 운영의 모습]



미리 판단을 한 상태에서 개발(행동)이 끝나갈 무렵에야 “이게 요구사항이었나?”라는 질문이 테스트 시점에 뒤늦게 터진다. 이게 요구사항이었나? 라는 질문이 판단 행동을 앞서지 못하고 맨 뒤로 밀려난 전형이다.

빠르게만 만들면 생기는 문제들



검증 근거 부재

이 코드가 정말 요구사항을 만족하는지 객관적으로 보여 줄 근거가 없다



추적 어려움

어떤 변경이 무엇을 왜 고쳤는지 나중에 따라가기 어렵다



위장된 통과

세션이 길어질수록 통과한 것처럼 보이지만 실제로는 아닌 코드가 는다

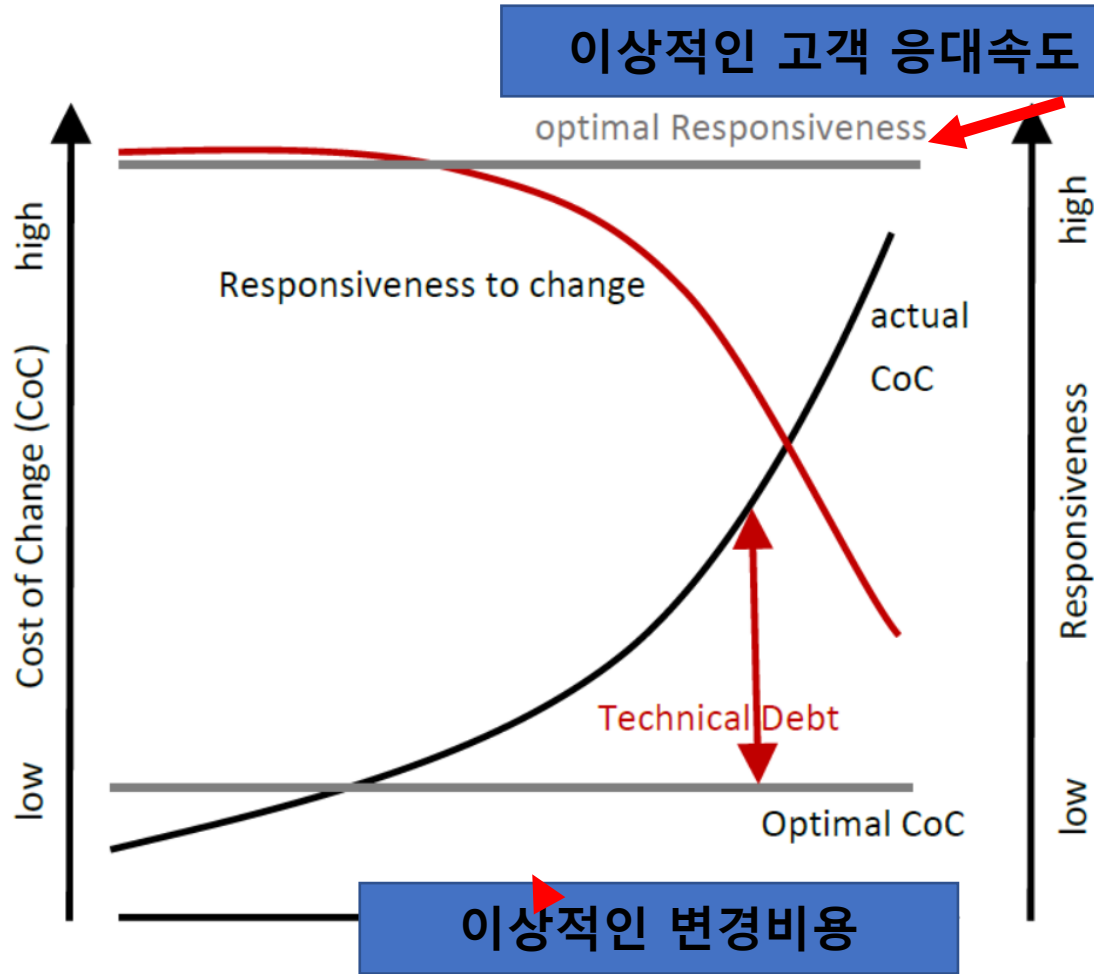
전통적 방식

기획서 → 개발자 해석 → 구현 → (뒤늦게) 문제 발견

Dual-Track TDD

실패 조건 합의 → RED 테스트 → 최소 구현(GREEN)

질문보가 판단응답이 앞선 경우와 변경 비용 — 시간이 지날수록



시간이 지날수록...

변경 비용

클린코드 ✕ → 기하급수적 증가

클린코드 ✓ → 완만하게 유지

응답 속도

클린코드 ✕ → 점점 느려짐

클린코드 ✓ → 빠른 응대 유지

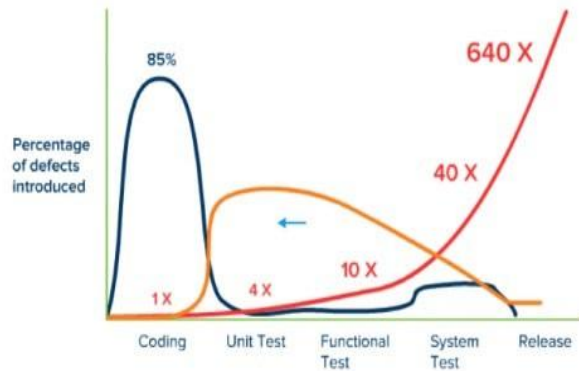
버그 발생

클린코드 ✕ → 수정 시 부작용

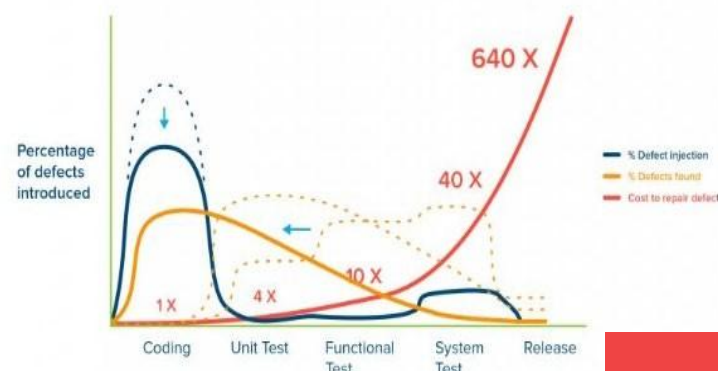
클린코드 ✓ → 테스트로 즉시 감지

테스팅의 비용 — 결함은 늦게 발견할수록 비싸다

결함은 늦게 발견할수록 수정 비용이 기하급수적으로 증가한다 (Cost of Defect).



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.



💡 TDD: 개발 시간 15~35% 증가 → 결함률 40~90% 감소 (Microsoft·IBM 연구)

개발의 다섯 단계 — 리서치에서 디버깅까지

1

리서치

요구·기존 코드·제약을 파악한다

2

계획

접근 전략과 작업 순서를 설계한다

3

코드

계획을 실제 구현으로 옮긴다

4

테스트

의도대로 동작하는지 검증한다

5

디버깅

실패 원인을 추적하고 고친다

에이전트와 함께하는 개발도 이 다섯 단계를 지난다 — 관건은 각 단계를 계약과 커밋으로 “추적 가능”하게 남기는 것(C2C). 바이브 코딩은 이 흐름을 “한 줄 요청”으로 뭉갬다.

단위 테스트 원칙 — F.I.R.S.T 원칙 (Python)

테스트가 없는 코드를 수정하면?
내가 고친 코드가 어떤 영향을 끼칠지 몰라 수정이 어렵다. 결국 코드는 썩는다.

테스트가 잘 갖춰진 코드를 수정하면?
잘못 고쳐도 테스트가 잡아 주니 편하게 수정한다. 리팩토링 자유도가 높아진다.

원칙	설명	Python (pytest)
Fast	테스트는 빨리 돌아야 한다	함수 단위 — ms 단위 실행
Independent	테스트는 서로 의존하면 안 된다	fixture로 초기화 (@pytest.fixture)
Repeatable	어떤 환경에서도 같은 결과	@pytest.mark.parametrize
Self-Validating	결과는 bool로 자동 판단	assert expected == actual
Timely	테스트는 실 코드 직전에 작성	TDD — 테스트 함수 먼저 작성

Right-BICEP & CORRECT — 무엇을 테스트할 것인가 (Python)

Right-BICEP

R — Right	결과가 옳은가? <code>assert add(5,5)==10</code>
B — Boundary	경계조건? <code>sys.maxsize, 0, None</code>
I — Inverse	역관계? <code>sorted</code> →역순 후 원본
C — Cross-check	다른 수단으로 교차 검증?
E — Error	에러 조건? <code>pytest.raises</code>
P — Performance	성능 조건? 타임아웃 측정

CORRECT (경계 조건)

C — Conformance	형식 준수: 이메일·날짜·JSON
O — Ordering	정렬·순서: 오름차순 보장
R — Range	범위: 나이 0~150, 인덱스
R — Reference	참조 유효성: 없는 ID·삭제된 객체
E — Existence	존재: None·빈 문자열·0
C — Cardinality	개수: 최소 1, 최대 100
T — Time	시간: 만료·타임아웃·동기화

AAA 패턴 — 좋은 테스트의 구조 (Python)

모든 테스트는 Arrange(준비) → Act(실행) → Assert(검증) 3단계로 명확히 구분한다.

Python — pytest

```
def test_mean_score_is_calculated():  
    # Arrange — 준비  
    collection = ScoreCollection()  
    collection.add(5)  
    collection.add(7)  
  
    # Act — 실행  
    result = collection.arithmetic_mean()  
  
    # Assert — 검증  
    assert result == 6
```

AAA 단계	pytest	핵심 원칙
Arrange	fixture / 객체 생성	테스트 픽스처 분리
Act	메서드 호출 1줄	단 하나의 동작
Assert	assert 문	명확한 성공/실패

Write Better Tests — 5 Steps

1

Production 코드처럼

테스트 코드도 실제 코드처럼 다룬다 — 읽기 쉽고 유지보수 가능하게.

2

패턴으로 가독성

AAA 등 테스트 패턴으로 의도를 명확히 드러낸다.

3

불안정 테스트 피하기

시간·랜덤·외부 의존을 제거해 Flaky를 없앤다.

4

적절한 수준 선택

단위·통합·E2E 중 검증 목적에 맞는 수준을 고른다.

5

Test Double 활용

핵심 외부 의존만 Fake/Mock으로 격리, 나머지는 실제로.

+

Test Smell 경계

다섯 원칙이 무너지는 지점이 곧 다음 장의 6대 냄새.

Test Smell — 6가지 대표 유형

Mystery Guest

X 외부 파일·DB 직접 의존 → 실패 원인 불명확

✓ 데이터 인라인 / Fake 객체로 격리

Assertion Roulette

X assert 여러 개 → 실패 위치 불명확

✓ SCOPED_TRACE + 실패 메시지 추가

Setup Bloat

X SetUp이 비대 → 의도 파악 어려움

✓ 각 테스트에 필요한 셋업만

Flaky Test

X 시간·랜덤·전역 상태 의존 → 실행마다 결과 다름

✓ MockClock/MockRandom · SetUp 독립

Eager Test

X 한 테스트에 기능 여럿 → SRP 위반

✓ 한 TEST = 한 관심사로 분리

Over-mocking

X 모든 것을 Mock → 실제 동작 미검증

✓ 핵심 외부 의존만 Mock, 나머지 실제

핵심 원칙 — 테스트 코드도 Production 코드처럼: 읽기 쉽고, 빠르고, 독립적이어야 한다. AAA 3분 할이 무너지는 곳에서 이 냄새들이 자랍니다.

테스트 케이스란 무엇인가

테스트 케이스(Test Case)란 요구사항을 준수하는지 검증하기 위하여 테스트 조건, 입력값, 예상 출력값 및 수행 결과 등 테스트 조건을 명시한 것이다.

테스트 케이스 = 특정 조건 + 입력값 + 기대 결과

실행 사전 조건

테스트를 수행하기 위한 전제 상태

입력값의 묶음

특정 프로그램 경로를 실행하도록 설계된 입력
집합

기대 결과·사후 조건

요구사항과 일치하는지 판정할 예상 출력

테스트 케이스 작성 절차 — 사칙연산 예 (Python)

테스트단계	입력값	예상값	중요도	성공/실패
1 + 10	1, 10	11	상	성공
0 + 1	0, 1	1	상	성공
0 ÷ 0	0, 0	ZeroDivisionError 발생	상	성공
-1 + (-10)	-1, -10	-11	중	성공

테스트 환경

Python 3.11+

pytest 8.x

VS Code + 통합 터미널

운영체제: Windows / macOS / Linux

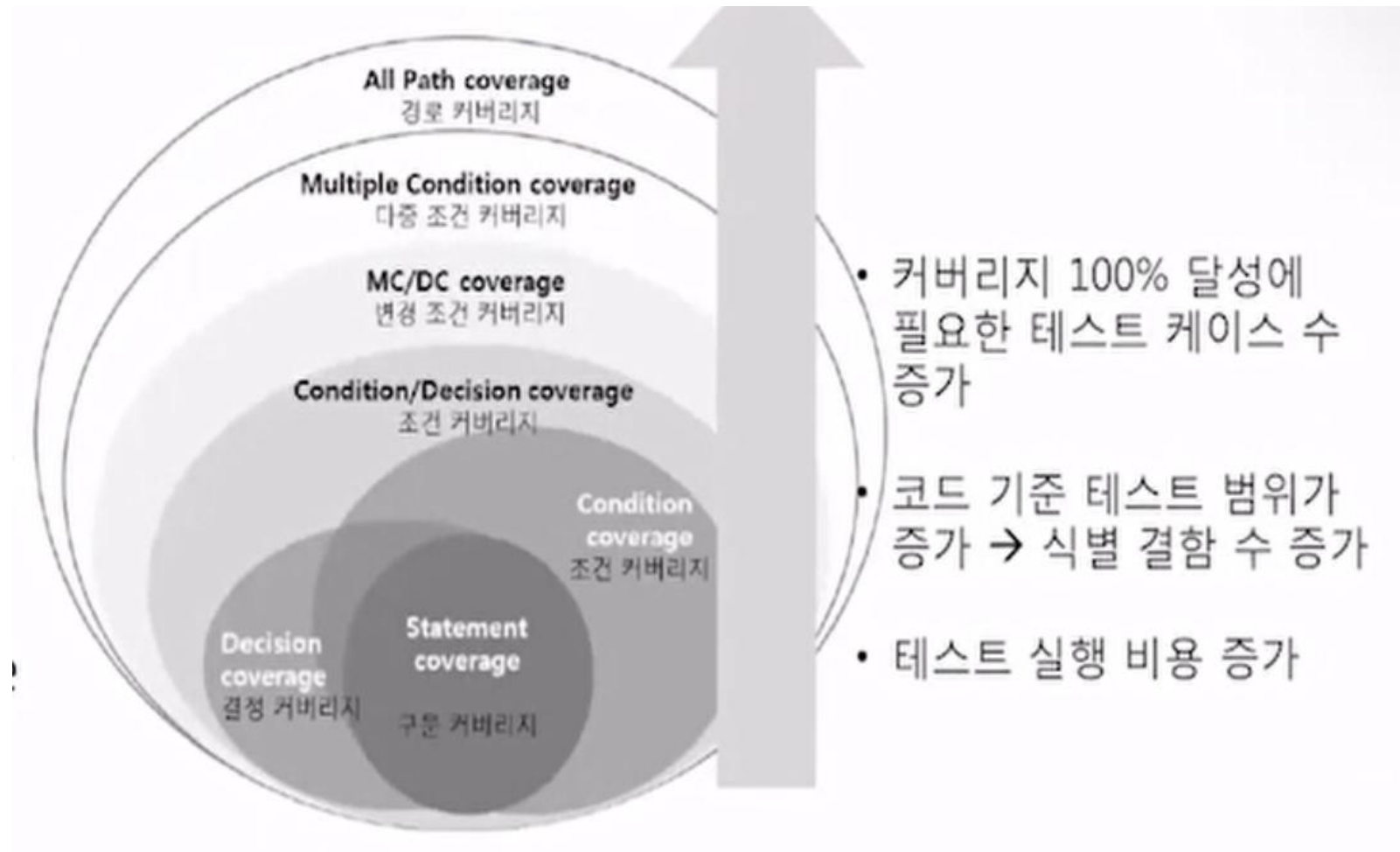
```
import pytest
from src.utils.math_utils import add, divide

def test_add_positive():    assert add(1, 10) == 11
def test_divide_by_zero():
    with pytest.raises(ZeroDivisionError):
        divide(0, 0)
```

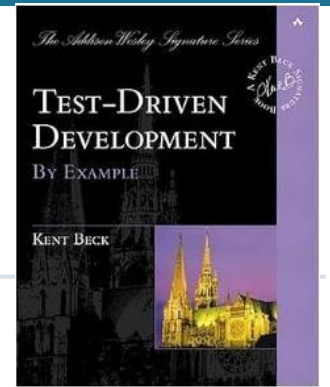
테스트 방법론-테스트 목적을 바탕

방법론	방법론 설명	테스트 방법론 사례
동적 테스트	프로그램 실행을 요구하는 테스트	화이트박스, 블랙박스
정적 테스트	프로그램 실행 없이 구조를 분석하여 논리성 검증	코드 검사, 워크스루
화이트박스 테스트	프로그램 내부 로직을 보면서 테스트 (구조 테스트)	구조 테스트, 루프 테스트
블랙박스 테스트	프로그램 외부 명세를 보면서 테스트 (기능 테스트)	동등분할, 경계값분석, 원인결과 그래프/오류예측
하드웨어 테스트	하드웨어의 성능과 신뢰성 테스트	부하테스트, 스트레스 테스트
소프트웨어 테스트	이해관계자들에게 시험 대상 제품 또는 서비스의 품질에 관한 정보를 제공하는 조사 과정	인간은 코드의 모든 결함을 찾는 것이 불가능하다.

Test Coverage



TDD 한 사이클 — RED · GREEN · REFACTOR (Python)



RED

실패하는 테스트를 먼저

만지는 것 · tests/ 만 — 구현(src/)은 건드리지 않는다

끝나는 신호 · 테스트가 “의도대로” 실패

GREEN

최소 구현으로 통과

만지는 것 · src/ 만 — 요구사항에 없는 기능은 금지

끝나는 신호 · 테스트가 전부 통과

REFACTOR

동작 그대로 구조 정리

만지는 것 · 중복 제거 · 이름 정리 — 계약 불변

끝나는 신호 · 여전히 GREEN 유지

```
# RED — tests/test_calc.py
def test_add(): assert add(2, 3) == 5
```

```
# GREEN — src/calc.py
def add(a, b): return a + b
```

혼자 도는 5단계 RED → GREEN 루프



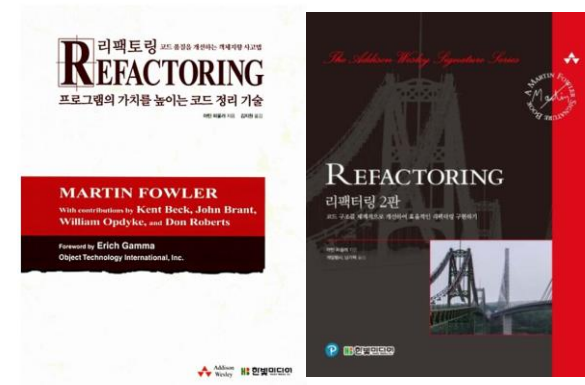
반복의 원칙 — 한 번에 요구사항 하나. RED→GREEN→커밋을 마치기 전에 다음으로 넘어가지 않는다. 실패 로그는 매번 새로 확인한다.

Dual-Track — 두 트랙으로 나눠 돌린다

한 화면짜리 앱이라도 “계산 규칙(Logic)”과 “화면 동작(UI)”은 성격이 다르다.

구분	Logic 트랙	UI 트랙
검증 대상	순수 함수(입력→출력) 계산 규칙	화면 렌더링·사용자 입력 흐름
이번 실습	subtotal · 할인 · VIP 계산	주문 폼 · 최종 금액 표시
속도/안정성	매우 빠르고 결정적	상대적으로 느리고 환경 영향
도구	pytest 단위 테스트	Flask test_client · Simple Browser · 브라우저 MCP

Code Smell & Refactoring



24가지 코드 냄새 · 리팩토링 기법 · 클린코드 원칙 · SOLID · 정적 분석 (Python). 외부 동작을 바꾸지 않으면서 내부 구조를 개선한다.

24가지 코드 냄새 (Fowler)

1 이해하기 힘든 이름	2 중복 코드	3 긴 함수	4 긴 매개변수 목록
5 전역 데이터	6 가변 데이터	7 뒤엎힌 변경	8 산탄총 수술
9 기능 편애	10 데이터 뭉치	11 기본형 집착	12 반복되는 Switch
13 반복문	14 정의 없는 요소	15 추측성 일반화	16 임시 필드
17 메시지 체인	18 중재자	19 내부자 거래	20 거대한 클래스
21 대안 클래스들	22 데이터 클래스	23 상속 포기	24 주석

3의 법칙 (Rule of Three) — Python 예제

돈 로버츠(Don Roberts): ① 처음엔 그냥 한다 ② 두 번째 중복은 일단 진행한다 ③ 세 번째 반복되면 리팩토링한다.

✗ 세 번째 중복 (냄새)

```
class UserService:
    def get_user_full_name(self, first, last):
        return f'{first} {last}'

    def get_admin_full_name(self, first, last):
        return f'{first} {last}' # 중복

    def get_guest_full_name(self, first, last):
        return f'{first} {last}' # 세 번째!
```

✓ 공통 로직 추출

```
def format_full_name(first, last):
    return f'{first} {last}'

class UserService:
    def get_user_full_name(self, f, l):
        return format_full_name(f, l)
    def get_admin_full_name(self, f, l):
        return format_full_name(f, l)
    # 이름 관련 로직을 한 곳으로 위임
```

객체 지향 설계 5원칙 — SOLID

S	단일 책임 (SRP)	하나의 클래스는 하나의 책임만 가진다
O	개방-폐쇄 (OCP)	확장에는 열려 있고 변경에는 닫혀 있다
L	리스코프 치환 (LSP)	파생 클래스는 기반 클래스를 대체할 수 있어야 한다
I	인터페이스 분리 (ISP)	쓰지 않는 메서드에 의존하지 않는다
D	의존 역전 (DIP)	추상화에 의존하고 구체적인 것에 의존하지 않는다

SRP 예시 — 책임을 분리한다 (Python)

클래스는 단 하나의 책임만 가진다 — 변경의 이유가 하나여야 한다. 직렬화·표현 책임은 별도로 분리한다.

✗ SRP 위반

```
class Resume:
    def introduce_me(self): ...
    def get_grade(self): ...
    def to_json(self): ... # 직렬화?
    def to_xml(self): ... # 직렬화?
    def to_html(self): ... # 표현/UI?
```

✓ SRP 준수

```
class Resume:
    def introduce_me(self): ...
    def get_grade(self): ...

class Converter(Protocol):
    def convert(self, r: Resume) -> str: ...

class JsonConverter: # Converter 구현
    def convert(self, r): ...
```

OCP · LSP · ISP · DIP — Python 적용 요약

원칙	위반 신호	Python 해법
OCP	새 유형마다 if/elif 추가	추상 기반 클래스(ABC)·Protocol + 딕셔너리 디스패치
LSP	자식이 부모 계약을 깬 (fly 못하는 Ostrich)	공통은 Bird, 능력은 Flyable 인터페이스로 분리
ISP	쓰지 않는 메서드 강제 구현 (Penguin.fly)	Bird / FlyableBird로 인터페이스 분리
DIP	구체 클래스에 직접 의존 (EmailService)	MessageService 추상화 + 생성자 주입(DI)

Python에서는 `abc.ABC`·`typing.Protocol`로 추상화를 표현하고, 생성자 주입으로 의존을 역전한다. 다형성과 덕 타이핑이 SOLID 적용을 자연스럽게 해 준다.

좋은 네이밍의 원칙 (Python)

의도를 표현하라

❌ `d` # 경과 일수

✅ `elapsed_days`

그릇된 정보 피하기

❌ `account_list` # List 아닐 수도

✅ `accounts`

의미 있는 구분

❌ `a1, a2 / product_info`

✅ `source, destination`

검색 가능한 이름

❌ `if s == 5:`

✅ `WORK_DAYS = 5`

클래스 = 명사

❌ `class Manager / Processor`

✅ `class Customer / Account`

함수 = 동사

❌ `name() / age()`

✅ `get_name() / is_valid()`

Python 네이밍 & 스타일 — PEP 8

구분	규칙 (PEP 8)	예시
클래스	PascalCase	CalculatorService
함수 / 메서드	snake_case	calculate_total()
변수	snake_case	elapsed_time
상수	UPPER_SNAKE_CASE	MAX_SIZE
비공개 멤버	선행 밑줄	_internal_state
모듈 / 패키지	짧은 소문자	cart, utils.math_utils
들여쓰기	스페이스 4칸	탭 대신 스페이스
최대 라인 길이	79 (docstring 72)	black / ruff 로 자동 정렬

 **VS Code:** ruff / black / flake8 로 스타일을 자동 강제할 수 있다.

함수 설계 — 작게, 하나만, 한 수준으로

1 작게 만들어라

20줄 이내가 이상적. 들여쓰기 1~2단계 이내

2 한 가지만 해라

함수 이름으로 설명되는 한 가지 작업만

3 추상화 수준 통일

고수준(비즈니스)과 저수준(I/O)을 섞지 말 것

4 서술적 이름

길고 서술적인 이름이 짧고 의미 없는 이름보다 낫다

5 인수를 최소화

이상 0개, 최대 3개. 초과 시 매개변수 객체 고려

6 부수 효과 없애기

숨겨진 변경 금지. 명령과 조회를 분리(CQS)

디미터 법칙 — 최소 지식 원칙 (Python)

Law of Demeter — 낯선 이에게 말 걸지 않는다. 연쇄 호출(.으로 이어지는 기차)은 결합도의 냄새다.

❌ 나쁜 예 (위반)

```
city = order.customer \
      .address \
      .city
```

- order→customer→address→city 내부 구조를 다 안다
- 구조가 바뀌면 호출부가 전부 깨진다

✅ 좋은 예 (준수)

```
city = order.customer_city()
# Order 내부에서 위임 처리:
# return self._customer.address.city
```

- 외부는 Order 하나만 알면 된다
- 내부 구조 변경이 호출부에 새어 나가지 않는다

위임 메서드로 감싸 내부 구조를 숨긴다 — 객체는 자신의 직접 이웃하고만 대화한다.

Clean Error Handling (Python)

원칙	나쁜 코드 ❌	좋은 코드 ✅
오류 코드 대신 예외	if delete_page(p) == E_OK:	try: delete_page(p) / except:
None 반환하지 않기	return None # 책임 떠넘김	빈 리스트 · 명시적 예외 반환
None 전달하지 않기	do_something(None)	가드 절 · Optional 명시
예외에 의미 담기	raise Exception("error")	raise StorageError(...) from e

```
# 예외 체이닝으로 원인 보존 (raise ... from e)
def read_config(path):
    try:
        return _parse(path)
    except OSError as e:
        raise StorageError(f'설정 읽기 실패: {path}') from e
```

코드스멜 주요 유형과 처방

중복 코드

같은 코드 두 곳 이상 → 함수 추출 / 공통 유틸

긴 함수

20줄 초과 → 기능별 분리

거대한 클래스

너무 많은 책임 → SRP 적용, 분리

긴 매개변수 목록

3개 초과 → 매개변수 객체 도입

죽은 코드

미사용 코드 → 즉시 삭제 (Git이 기억)

매직 넘버

의미 불명 값 → 이름 있는 상수로 추출

깊은 중첩

3단계 이상 → Early Return / 함수 추출

부적절한 이름

a, tmp, data → 의도를 드러내는 이름

정적 분석으로 코드스멜을 자동 탐지 (Python)

정적 분석은 실행 없이 소스코드를 분석해 버그·스타일 위반·복잡도 문제를 찾아낸다.

도구	탐지 내용	연동 방법
Pylance	실시간 타입·문법 경고	VS Code Python 확장 내장
ruff	스타일·버그·복잡도 (초고속)	ruff check . / pre-commit
flake8	코딩 규칙 (PEP 8·복잡도)	flake8 + 설정 파일
mypy	정적 타입 검사	mypy src
bandit	보안 취약 패턴	bandit -r src
pytest-cov	테스트 커버리지 측정	pytest --cov=src

ruff check . && mypy src && pytest --cov=src — 한 파이프로 정적 분석을 자동 실행한다.

QA를 위한 프롬프트 엔지니어링 — 5요소

막연한 “고쳐줘”는 범위가 폭주한다. QA 프롬프트는 아래 5요소를 명시할수록 결과가 안정적이다.

- | | | |
|---|-------|------------------------|
| 1 | 역할/관점 | “보안 리뷰어 관점에서” |
| 2 | 맥락 | “calc 함수만, 동작 변경 없이” |
| 3 | 작업 | “스멜을 심각도 순으로 나열” |
| 4 | 제약 | “파일 수정 말고 보고만” |
| 5 | 출력 형식 | “표: 위치 문제 위험 제안” |

출력 형식을 지정하면 결과를 그대로 QA 리포트·Skill 절차로 재활용할 수 있다 — 형식은 자동화의 입력이 된다.

작업별 프롬프트 패턴 — 진단 · 리뷰 · 리팩터 · 테스트

진단

«대상»의 코드 스멜·중복·복잡도를 찾아줘. 수정 말고 [위치|유형|근거|난이도] 표로, 우선순위 순으로 정리해줘.

리뷰

방금 변경을 «보안|회귀|성능» 관점으로만 리뷰해줘. 동작이 바뀔 지점과 그걸 잡을 테스트가 없는 부분을 우선순위로.

리팩터

«함수»의 외부 동작을 바꾸지 말고 내부 구조만 개선해줘. 먼저 계획을 보여주고 승인 후 적용, 매 단계 후 테스트 실행.

테스트

«함수»의 테스트 갭을 찾아줘(경계·예외·중복). 빠진 케이스를 먼저 보여주고, 합의된 케이스만 테스트로 추가해줘.

범위 폭주를 막는 한 줄 — “먼저 계획을 보여주고 승인 후 진행. 요청하지 않은 변경은 하지 마.”

미니 체험 ④ — Unit Test

RED — 의도된 실패

프롬프트: “tests/test_calc.py 에 test_add 테스트만.
src/calc.py 는 만들지 마.”

“add가 없으니 pytest는 의도된 실패(RED).”

GREEN — 최소 통과

프롬프트: “방금 테스트를 통과시키도록 add(a,b)를
가장 단순하게 구현해줘.”

“1 passed — 이 테스트는 이제 안전자산.”

미니 체험 ⑤ — Code Smell & Refactoring



Concept-to-Code (C2C) 딜리버리 프레임워크 와 Traceability

실패 없는 소프트웨어를 위한 해석학적 접근. 개념·불변식에서 코드까지 연결고리를 끊지 않는 개발.

C2C의 정의와 4계층 검증 구조

C2C(Concept-to-Code) — 개념·불변식에서 코드까지 연결고리를 끊지 않는 것. 임의의 구현 줄에서 위로 올라가면 반드시 불변식에, 임의의 불변식에서 아래로 내려가면 테스트·구현 줄에 닿는다 — 이 양방향 도달 가능성이 곧 C2C.

계층	이름	질문	ECB 대응
Layer 1	Invariant (불변식)	무엇이 절대 바뀌면 안 되는가	수학적 진실
Layer 2	UI Contract Test	UX가 올바르게 반응하는가	Boundary
Layer 3	Domain Test	비즈니스 규칙이 작동하는가	Control
Layer 4	Implementation Unit	단위 알고리즘이 정확한가	Entity

△ ECB는 MVC·Spring Controller와 다르다 — 분석 개념 도구이지 코딩 규칙이 아니다 (Ivar Jacobson, OOSE 1992). Layer 1(불변식)은 사슬의 정점 — 그 불변식은 어디서 오는가? 이 질문이 3장(제품 계약)으로 이어집니다.

계약 ID(추적의 못)와 추적성 매트릭스

RED

테스트 이름·주석에 계약 ID

GREEN

구현 줄마다 계약 ID 주석

COMMIT

커밋 메시지에도 계약 ID

금지

ID 없는 동작은 구현하지 않는다

계약 ID	의미	검증 테스트	구현 위치
INV-1	subtotal = $\Sigma \text{price} \times \text{qty}$	test_subtotal_sums_price_times_qty	subtotal() return
INV-2	경계 포함 문턱 할인	test_threshold_applies_at_50000	apply_threshold_discount()
INV-3	문턱→VIP 순서·금액	test_vip_order_and_value	final_total()
INV-4	$0 \leq \text{final} \leq \text{subtotal}$	test_discount_never_increases	final_total() return
E-1 / E-2	None→TypeError / 음수→ValueError	boundary::test_none / _negative	subtotal() 가드/루프

구현에는 있으나 매트릭스에 없는 코드 줄 = “정체불명 코드” → 제거하거나 계약화. 켄트 벡이 꼽은 AI의 첫 번째 경고 신호(과잉 구현)를 원천 차단합니다.

Dual-Track TDD와 ARRR 실행 리듬

단계	의미	TDD	완료 신호
A — Ask	테스트로 시스템에 질문한다	RED	의도된 FAIL
R — Respond	통과하는 최소 코드로 답한다	GREEN	전부 PASS
R — Refine	중복 제거·이름 정리로 다듬는다	REFACTOR	계약 불변·GREEN
R — Repeat	다음 계약 ID로 다음 질문	다음 RED	회귀(pytest -q)

이중성 ① — 프로세스 수준

Discovery(무엇이 옳은가) || Delivery(옳게 만든다). Discovery의 계약·골든셋이 Delivery의 픽스처가 되고, Delivery의 계측이 Discovery의 다음 가설이 된다.

이중성 ② — 테스트 아키텍처

Track A(UI/Boundary) || Track B(Domain/Entity). 각 트랙에서 ARRR를 돌리며 4계층의 Layer 2와 Layer 3~4를 각각 담당한다.

용어 주의 — 네 글자 ARRR(Ask·Respond·Refine·Repeat)은 실행 리듬, 다섯 글자 AARRR(Acquisition...Revenue)은 그로스 퍼널 지표. 별개의 개념입니다.

ARRR 예제 — 숫자 더하기 계산기 (Python)

빈 문자열 → 0, 단일 숫자 → 그 값, 쉼표 구분 합계. 이후 세미콜론 구분자를 Repeat로 추가한다.

A · Ask (RED)

```
def test_empty_returns_zero():
    assert add('') == 0
def test_single_number():
    assert add('5') == 5
def test_two_numbers():
    assert add('3,5') == 8
```

R · Respond (GREEN)

```
def add(numbers):
    if not numbers:
        return 0
    return sum(
        int(n) for n in numbers.split(',')
```

R · Refine (REFACTOR)

```
# 구분자 분리 로직을 함수로 추출(SRP)
def _split(numbers): ...
```

R · Repeat (새 RED)

```
def test_semicolon_delimiter():
    assert add('3;7') == 10 # 새 요구사항
```

Dual-Track TDD 5단계 사이클 — 두 트랙의 맞물림

단계	Discovery 트랙	Delivery 트랙	ARRR	신호
① 가설 수립	계약·골든셋·목표 수치 정의, ID 부여	(대기)	—	계약 표
② Red	가설을 검증 가능한 조건으로 구체화	ID마다 실패 테스트(tests/만)	Ask	의도된 FAIL
③ Green	구현이 가설을 만족하는지 1차 확인	통과 최소 구현, 구현 줄에 ID 주석	Respond	전부 PASS
④ Refactor	평가 기준이 정밀해질 지점 식별	중복 제거·SSOT 추출, 계약 불변	Refine	여전히 GREEN
⑤ 합류/반복	골든셋·AARRR 지표로 전체 검증	E2E 스모크 + 다음 ID로 회귀	Repeat	매트릭스 완성

닫힌 루프 — Discovery의 산출물이 Delivery의 테스트 픽스처가 되고, Delivery의 계측이 Discovery의 다음 가설이 됩니다. 이것이 프로세스 수준의 Dual-Track입니다.

TDD · 리팩터링 · Tidy First — 계승하는 규율

단계	이때 만지는 것	끝나는 신호
RED	tests/ 만	테스트가 "의도대로" 실패
GREEN	src/ 만	테스트가 전부 통과
REFACTOR	src/ (필요 시 tests/)	여전히 통과 + 더 깔끔

TDD 세 원칙

① 실패를 먼저 본다 ② 최소로 통과시킨다(과잉 구현 금지) ③ 통과 후 정리한다

Tidy First 3규칙

① 구조 변경과 동작 변경을 한 커밋에 섞지 않는다 ② 둘 다 필요하면 구조 변경 먼저 ③ 전후로 테스트를 돌려 동작 불변 확인

코드 스멜 대표 처방 — 중복 코드→함수 추출 · 긴 함수→Extract Function · 불명확한 이름→Rename · 매직 넘버(50000, 0.9)→상수로 추출(SSOT).

Augmented Coding — “씨앗을 먹지 마라”

“AI에 코딩을 무심코 맡기는 것은, 농부가 배고프다고 다음 해 심을 씨앗까지 먹어 치우는 것과 같다.” — 켄트 벅

	Vibe Coding	Augmented Coding
태도	돌아가면 OK, 결과를 안 본다	주도권을 쥐고 결과에 책임진다
관심사	동작만	동작 + 테스트 + 설계 품질
AI와의 관계	맡기고 기다린다	감시하며 필요하면 개입·중단

△ AI가 결길로 새는 경고 신호 — ① 요청하지 않은 기능(과잉 구현) → “ID 없는 동작은 구현하지 않는다”가 차단 ② 테스트를 속이는 정황(비활성화·삭제·수정) → “테스트 약화 금지”가 차단. 두 신호는 2장 하네스에서 금지(Don't) 5조항으로 성문화됩니다.

프로젝트 수준 — 13 STEP · 4개 페이즈 조감도

1

기획·아이템 선정

STEP 1~5
Self-Audit·Job검증(JTBD·Mom
Test·VPC)·아이템 선택(ICE)

2

제품 설계·지표

STEP 6~8
제품 계약
(INV/E/AC+L0~L2)·AARRR 역산
·PRD 확정

3

엔지니어링·구현

STEP 9~12
대표 모듈 시연(AARRR 완주)·하네스
구축·모듈별 8단계 반복

4

평가·회고

STEP 13
골든셋 평가·AARRR 실측·추적성 매
트릭스 점검·KPT

모듈 수준의 8단계 골격이 프로젝트 수준에서 13 STEP으로 확장됩니다. 오늘 실습 범위 — 나머지는 프로젝트 가이드(D2)의 Magic Square로 이어집니다.

Hands-on Exercise — 장바구니 할인 계산기

Hands-on Exercise. “장바구니 할인 계산기”를 TDD로 완성한다. Logic 트랙(subtotal→문턱 할인→VIP·pytest) + UI 트랙(주문 폼→최종 금액·Flask).

실습의 뼈대 — Discovery → Delivery 8단계 골격

Discovery — 코드 작성 전에 끝낸다

- ① 목표 — 보장할 것 + ECB 역할·C2C 계층·트랙 좌표를 못 박는다
- ② PRD(실패 조건) — 기능 목록이 아니라 “무엇이 실패인가”를 L0~L3로
- ③ 계약 식별+ID — INV-*·E-*에 ID 부여 (추적의 못)
- ④ 스케레톤 — 시그니처 고정 + TODO(계약 ID). 완성 코드는 감춘다

Delivery — ⑤~⑧이 ARRR 사이클

- ⑤ A = Ask(RED) — ID마다 실패 테스트를 tests/에만. src/ 금지
- ⑥ R = Respond(GREEN) — 실패 로그 → 최소 구현. 구현 줄에 ID 주석
- ⑦ R = Refine(REFACTOR) — 중복 제거·SSOT. 동작·계약 불변
- ⑧ R = Repeat — 다음 계약 ID로 다음 RED + 회귀(pytest -q)

절대 곧바로 코딩하지 않습니다. ④ 스케레톤에서 완성 코드를 감추는 것은 교육 장치이자 추적성 장치 — 학습자와 AI 모두 “계약→테스트→구현 줄” 순서로만 코드에 도달합니다.

무엇을 만드나 — 장바구니 할인 계산기, 두 트랙으로

Track B — Logic (Entity)

금액·할인 계산(순수 함수) — 합계 → 문턱 할인 → VIP 할인

INV-1 · INV-2 · INV-3 · INV-4 · E-1 · E-2

tests/entity/

상류(3장) — Mom Test 메모에서 제품 계약(INV/E/AC+ 증거 수준)을 도출

Track A — UI (Boundary)

주문 폼 웹페이지(Flask) — 수량·VIP 입력 → 최종 금액 표시

UC-1 · UC-2 · UE-1 (+ 계측 M-1 · M-2)

tests/boundary/

하류(6장) — AARRR 계측 인터페이스를 계약ID별 AARRR로 구현

ECB 방향 단속 — Boundary는 Entity를 import 할 수 있지만, Entity는 Flask를 절대 import 하지 않습니다. 이 방향이 깨지면 Layer 4 검증이 외부 의존에 오염됩니다.

파이썬 프로젝트 초기화 — 가상환경과 패키지

터미널

```
# 프로젝트 폴더 만들기
mkdir codex-c2c-cart && cd codex-c2c-cart
code . # VS Code로 이 폴더 열기

# 파이썬 가상환경 생성/활성화
python -m venv .venv
source .venv/Scripts/activate # Windows Git Bash
# macOS/Linux: source .venv/bin/activate

# 실습 패키지 설치 및 고정
pip install flask pytest
pip freeze > requirements.txt
```

✓ 성공 신호

- 프롬프트 앞에 (.venv)가 보인다

- requirements.txt가 생성됐다

자주 하는 실수

가상환경을 켜지 않고 pip install → pytest가 전역에 깔려 “왜 import가 안 되지?”로 이어진다.

실습 세션을 새로 열 때마다 가상환경 활성화가 첫 명령입니다 — 4장·5장 시작 전 체크포인트에 매번 등장합니다.

GitHub 저장소 연결 — 처음부터 Git을 켜다

터미널

```
git init
# GitHub 웹에서 빈 저장소 생성 후:
git remote add origin <저장소 주소>
git add . && git commit -m "chore: 초기 스캐폴딩"
git push -u origin main
```

앞으로 남길 커밋의 형식

test(entity): INV-1 ... [RED]

feat(entity): INV-1 ... [GREEN]

refactor(entity): SSOT [REFACTOR]

docs: 제품 계약 SSOT (INV-1~4)

형식: <타입>(<계층>): <계약 ID> <무엇> [단계]

커밋 메시지에 계약 ID를 적는 규율(2.8절)이 “버전 이력 = 추적성 매트릭스의 시간축 사본”을 만듭니다. git log --oneline만으로 어느 계약을, 어느 단계에서 다뤘는지가 추적됩니다(7.2절에서 재 확인).

실습 환경 최종 체크리스트 — 모두 OK면 2장으로

- ☐ 터미널에서 codex가 실행되고 ChatGPT 계정으로 로그인되어 있다
- ☐ VS Code에서 Codex IDE 확장(VS Code) 채팅·Agent 탭이 열린다
- ☐ node --version이 20 이상, python --version이 3.11 이상을 출력한다
- ☐ pip install flask pytest가 오류 없이 끝났고, 프롬프트 앞에 (.venv)가 보인다
- ☐ git status가 정상 동작하고, 원격 저장소가 연결돼 있다

하나라도 체크되지 않으면 지금 해결하고 넘어가세요. 실습 중 환경 문제로 막히는 시간이 가장 아깝습니다 — 이제 이 폴더 위에 2장에서 하네스를 세웁니다.

02

하네스 (Harness) — 재현 가능한 테스트 골격

2장 · 프롬프트 · 컨텍스트 · 하네스의 3수준. ECB 폴더 · AGENTS.md · pytest 하네스로 “지도”를 만든다.

3수준 사다리 — 프롬프트 c 컨텍스트 c 하네스

①

프롬프트 엔지니어링

지시문 자체 — 역할·작업·출력 형식(개별 요청의 품질).

실체: 세션에 입력하는 한 번의 요청·프롬프트 박스

②

컨텍스트 엔지니어링

모델이 보는 정보 환경 — “지금 상황”을 정확히 주기.

실체: @파일 참조·AGENTS.md 계층 병합·테스트 실패 로그

③

하네스 엔지니어링

실행 골격 전체 — 세션을 초월해 상시 작동하는 구조물.

실체: pytest 하네스·AGENTS.md·config.toml·커맨드·MCP·금지 규칙

Agent = Model + Harness — LLM은 엔진일 뿐. 컨텍스트의 왕 = 테스트 실패 로그. 실패 로그 + 최소 구현 요구 = Respond의 표준형이며, 이것이 과잉 구현을 억제합니다.

프롬프트 5트랙 — 입력 채널 매핑

트랙	언제 쓰는가	입력 채널	프롬프트 패턴(요지)
Discovery	계약·정책·지표 결정	codex 세션 / 채팅	매트릭스로 제안하고 각 칸의 근거를 한 줄씩
Red(Ask)	새 계약 ID 착수(항상 먼저)	커스텀 /tdd-red	INV-*를 검증하는 테스트만. tests/만 수정
Green(Respond)	Red 테스트 통과	codex 세션 (Agent)	[실패 로그] 최소 구현만. 구현 줄에 ID 주석
Refactor(Refine)	전부 GREEN 이후 정리	plan 모드 리뷰 후	동작·계약 불변으로 SSOT 추출. 깨지면 멈춰
운영/그로스	지표 설계·퍼널 진단	codex 세션	최대 이탈 단계, 원인 가설, 다음 실험 한 가지

모든 프롬프트는 이 다섯 트랙 중 하나입니다. 지금 내가 어느 트랙에 있는지 모른 채 던지는 요청이 바로 바이브 코딩입니다. (Chat=설계 대화 · Inline Edit=단일 파일 RED · Composer/Agent=다중 파일 GREEN)

ECB로 나눈 폴더 + 작업 규칙의 “지도”

프로젝트 구조 (ECB 계층별)

codex-tdd-cart/

AGENTS.md 작업 규칙, 컨텍스트 (TDD 정책의 지도)

pytest.ini · conf/test.py 하네스 설정 · import 경로

src/ cart.py Entity — 금액, 할인 순수 함수 (Track B)

src/ app.py Boundary — Flask 주문 폼 (Track A)

tests/ entity/ Track B 계약 검증 (INV-*)

tests/ boundary/ Track A 계약 검증 (E-*, UC-*)



AGENTS.md — TDD 규칙을 “지도”로

저장소 루트에 두면 Codex CLI·Code Assist가 매 작업마다 자동으로 읽는 컨텍스트 파일. 백과사전이 아니라 “짧은 지도”로 씁니다.

작성 5원칙

- 1 짧게 — 30줄이 300줄보다 낫다
- 2 테스트 “명령”을 명시한다
- 3 “성격”이 아니라 “행동”을 적는다
- 4 금지(Don't) 목록을 둔다
- 5 자세한 건 docs/로, 여기엔 “어디를 보라”만

프로젝트 구조 만들기 — 폴더가 곧 계층 선언

목표 구조

```
codex-c2c-cart/
├── AGENTS.md           # 하네스의 "지도"
├── .codex/
│   ├── config.toml    # 승인 정책 · 도구 · MCP
│   └── prompts/       # 커스텀 슬래시 명령 (.md)
├── docs/PRD.md        # 제품 계약 SSOT — 3장
├── pytest.ini         # 테스트 하네스 (트랙 마커)
├── conftest.py        # src 임포트 경로
├── src/  cart.py(Entity) app.py(Boundary) metrics.py
└── tests/  entity/  boundary/  golden/
```

한 번에 만들기

```
mkdir -p src tests/entity tests/boundary
        .codex/prompts docs
```

```
touch src/__init__.py src/cart.py
```

테스트를 ECB 계층별로 나눠 두면 “어느 계층의 계약을 검증하는지”가 폴더만 봐도 드러납니다. cart.py는 비워 둡니다 — RED에서 실패를 보기 위해.

src/cart.py를 일부러 비워 두는 것이 4장 RED의 출발점입니다 — 실패를 눈으로 보지 않은 테스트는 검증하는 테스트인지 알 수 없습니다.

Test Harness 구성(pytest) — 트랙 마커 분리

pytest.ini

```
[pytest]
testpaths = tests
python_files = test_*.py
markers =
    entity:    Track B - 불변식 (INV-*)
    boundary:  Track A - 입력/UI (E-*, UC-*)
    slow:      외부 호출 통합/E2E
    golden:    회귀 기준선 (동작 동일성)
```

conftest.py

```
# 프로젝트 루트를 import 경로에 추가
import os, sys
sys.path.insert(0,
    os.path.dirname(__file__))

# ModuleNotFoundError: 'src'
# → 이 파일이 루트에 없어 생기는
# 1순위 오류
```

✓ pytest 실행 시 “no tests ran” = 정상(에러 아님) · tests/entity/와 tests/boundary/ 폴더 존재. 외부 의존은 slow 마커로 분리 — 이 분리가 Agent의 자동 반복(Repeat) 루프의 전제입니다.

AGENTS.md — 작업 규칙을 “지도”로

AGENTS.md

```
# Repository instructions
## 구조 (ECB)
- cart.py: Entity, Flask import 금지
- app.py: Boundary, cart.py 재사용
## 계약 (C2C)
- SSOT는 docs/PRD.md
- ID 없는 동작은 구현하지 않는다
## 워크플로 — ARRR
- "실패하는 테스트"가 항상 구현보다 앞선다
```

금지 5조항(Don't) — 하네스의 핵심

- ① 테스트 수정·삭제·skip으로 우회 금지
- ② 요청하지 않은 기능 금지(과잉 구현)
- ③ 함수 시그니처 임의 변경 금지
- ④ ID 없는 동작 구현 금지
- ⑤ 골든 마스터 파일 임의 재생성·수정 금지

작성 5원칙 — ① 짧게(30줄이 300줄보다) ② 테스트 “명령”을 명시 ③ “성격”이 아니라 “행동” ④ 금지 목록을 둔다 ⑤ 자세한 건 docs/로 미룬다. 규칙 변경 후 /memory refresh.

.codex/config.toml — 승인 정책과 도구 범위

approval_policy · sandbox_mode	무엇	언제
sandbox: read-only	읽기 전용 — 분석만	Review·스멜 탐지(7.3)
approval: untrusted	모든 수정·명령을 매번 승인	기본값(권장 시작점)
--full-auto	파일 편집만 자동 승인	GREEN 반복이 익숙해진 후
never (샌드박스 오프)	전부 자동 승인(주의)	격리된 실행 환경에서만

```
model = "gpt-5.5-codex"
approval_policy = "untrusted" # untrusted | on-request | never
sandbox_mode = "workspace-write" # read-only | workspace-write | danger-full-access
# AGENTS.md 를 지도로 읽음 · git push·rm 은 자동 승인 금지
```

안전 주의 — 본인이 쓰지 않은 AGENTS.md·config.toml·확장은 “신뢰할 수 없는 코드”처럼 다루세요. git push·배포·rm 같은 되돌리기 어려운 명령은 승인 요청 시 직접 승인합니다.

커스텀 커맨드 — ARRR를 슬래시 버튼으로

/tdd-red

계약 ID의 '실패하는' 테스트만 작성.
tests/만 수정, src/ 금지. 실패 로그 전체 표시. 첫 줄에 Phase: RED 선언.

/tdd-green

실패 로그를 통과시키는 '최소' 구현만.
시그니처 불변·구현 줄에 ID 주석·테스트 수정/skip 금지.

/tdd-refactor

동작·계약 불변으로 구조만 정리(중복 제거·SSOT). 전후 pytest·골든 게이트 확인. 없으면 "없음".

/golden-capture

GREEN 상태의 동작을 입력 그리드 위에 스냅샷으로 떠 회귀 기준선. RED에서는 캡처 금지 — 버그 고정.

.codex/prompts/*.md — 파일명이 곧 명령 이름. /tdd-refactor는 골든 게이트를 포함 — 리팩터 전후 pytest -m golden -q가 모두 GREEN이어야 하고, 깨지면 되돌립니다(골든 파일 수정은 금지 5조 위반).

심화 확장 통로 — MCP · Hooks (부록 G로 이동)

MCP — 외부 도구 연결

~/codex/config.toml의 [mcp_servers.*]에 서버를 등록하면 context7(최신 라이브러리 문서)·github(이슈·PR) 같은 외부 도구를 세션에 연결한다.

→ 설정 예시는 부록 G.1~G.2

Hooks — 감사 로그 자동 기록

after_tool_use 훅으로 도구 실행마다 “날짜|도구|사용자”를 감사 로그에 자동으로 남긴다. 실행권한(chmod +x) 없이는 조용히 실패한다.

→ 설정 예시는 부록 G.3~G.5

선택 심화 항목입니다 — 오늘 실습의 필수 경로는 아니지만, 실무 프로젝트에서 하네스를 확장하는 두 표준 통로입니다. 전체 설정은 부록 G에서 이어집니다.

staging-spec 브랜치와 3종 커밋

터미널

```
# 1단계 - 계약 묶음 1개 = 브랜치 1개
git switch -c staging-spec/entity-cart
# 2단계 - RED(테스트만, 실패 확인 후 커밋)
git add tests/entity/test_cart.py
git commit -m "test(entity): INV-1 ... [RED]"
# 3단계 - GREEN(최소 구현만)
git add src/cart.py
git commit -m "feat(entity): INV-1 ... [GREEN]"
```

판단 기준 — 매번 이것만

새 계약(INV-*,E-*) 하나?

→ staging-spec/... 새 브랜치

테스트만? → [RED]

구현만? → [GREEN]

구조만? → [REFACTOR]

Entity+Boundary 동시?

→ git worktree로 폴더 분리

버전 이력 자체가 추적성 매트릭스의 시간 축 사본이 됩니다 — git log --oneline만으로 “어느 계약을, 어느 단계에서” 다뤘는지 추적됩니다.

2장 마무리 — 하네스 완성 후, 3장으로

- ❑ AGENTS.md가 존재하고 구조(ECB)·계약(C2C)·금지 5조항이 담겨 있다
- ❑ pytest.ini·conftest.py가 존재하고 pytest -q가 오류 없이 실행된다
- ❑ .codex/config.toml에 승인 정책·허용 도구가 설정돼 있다
- ❑ .codex/prompts/*.md로 /tdd-red 등 커스텀 커맨드가 동작한다
- ❑ /memory show로 병합된 컨텍스트를 확인했고, staging-spec/entity-cart 브랜치가 있다

2장에서 하네스(harness)를 세웠습니다 — 3장부터는 이 위에서 실제 제품 계약을 설계합니다.

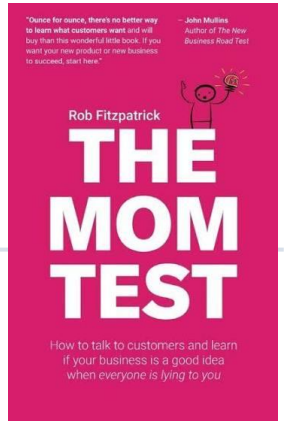
03

Discovery — Mom Test로 제품 계약을 캐낸다

3장 · 상류(Discovery). 검증 가능한 질문으로 계약을 도출해 docs/PRD.md에 SSOT로 고정한다.

The Mom Test 란?

The Mom Test(엄마 테스트)는 롭 피츠패트릭(Rob Fitzpatrick)이 저술한 동명의 책에서 유래한 개념으로, 사업 아이디어가 유효한지 검증하기 위해 고객과 진솔한 대화를 나누는 방법론



스타트업·제품 아이디어 검증에서 쓰이는 인터뷰 기법. 상대가 ‘좋은 사람이라서’ 거짓 긍정(칭찬·공감)을 해도 속지 않도록 질문하는 방법이다.

Mom Test = “사람의 말이 아니라 행동을 믿는 인터뷰 방법”

계약(INV·E·AC)의 원천은 “고객이 실제로 어떻게 행동했는가”다. Mom Test는 그 근거를 캐내는 도구다.

잘못된 질문 vs 올바른 질문

✖ 잘못된 질문

“제가 만든 설비 모니터링 앱 어때요?”

→ 칭찬만 들을 것

“이런 기능이 있으면 사용하시겠어요?”

→ 가짜 약속만 받을 것

“이 앱에 얼마 지불하시겠어요?”

→ 현실성 없는 답변만 받을 것

✔ 올바른 질문

“최근 설비 점검하며 가장 답답했던 순간은?”

→ 진짜 문제 발견

“지금은 그 상황을 어떻게 해결하고 계세요?”

→ 현재 우회 방법 파악

“그 방법의 어떤 점이 불편하세요?”

→ 개선 포인트 확인

핵심 원칙 3가지와 피해야 할 3가지

핵심 원칙 3가지

- 당신의 아이디어에 대해 말하지 마라
- 미래나 가정에 대해 묻지 마라
- 과거의 구체적인 행동에 대해 물어라

피해야 할 3가지 데이터

칭찬 (Compliments)

“정말 좋은 아이디어네요!” → 행동으로 이어지지 않음

가상의 약속 (Hypotheticals)

“있으면 꼭 쓸 것 같아요!” → 실제로는 안 씀

일반론 (Generics)

“보통 사람들은 좋아할 거예요” → 추측일 뿐

올바른 질문하기 — 과거·투자·해결 시도

과거의 구체적 사건

“지난주 설비 점검 때 어떤 일이 있었나요?”
“정확히 어떤 절차를 거치셨나요?”
“얼마나 시간이 걸렸나요?”

돈·시간 투자 확인

“현재 도구에 비용을 지불하시나요?”
“배우는 데 시간을 얼마나 쓰셨나요?”
“팀원들도 다 쓰고 있나요?”

해결 시도 이력

“이 문제를 해결하려 시도해 보셨나요?”
“어떤 도구들을 써보셨나요?”
“왜 계속 쓰지 않으셨나요?”

인터뷰 진행하기 — Mom Test 실전 (Codex 역할극)

실무에서는 인터뷰 메모를 스스로 만들어야 한다. Codex에게 질문지를 짜게 하고, 고객 역할을 맡겨 연습한다.

① 인터뷰 가이드 생성

당신은 Mom Test 규칙을 엄격히 적용하는 인터뷰 설계자다.

job: "주문 담당자가 할인 금액을 오류 없이 확정"

1. 과거 행동을 캐는 질문 8개(미래·의견 금지)
2. 각 질문 옆에 '캐내려는 계약 후보' 한 줄
3. 반드시 포함:
'절대 발생하면 안 되는 결과는?' → INV 후보
'잘못 입력하면 뭐가 일어났나?' → E 후보
4. 물으면 안 되는 유도 질문 3개를 반례로

② 역할극 — Codex가 고객이 된다

지금부터 당신은 온라인 쇼핑몰 주문 담당자다.

- 과거에 실제 겪은 일만 말한다.
해결책을 제안하지 않는다.
- 내가 유도 질문('~하면 되죠?')을 하면,
먼저 '유도 질문입니다'라고 지적하고
어떻게 바꿀지 알려준다.
- '인터뷰 종료' → [인터뷰 메모] 형식 정리.
없는 사실을 지어내지 않는다.
첫 질문을 기다린다.

인터뷰 메모를 계약으로 변환

[인터뷰 메모] — 실습 입력

- A 3개(12,000)+B 1개(30,000)=66,000. 5만원 넘으면 10% 할인. 기대 59,400 (엑셀 계산 10분)
- VIP는 문턱 할인 후 추가 5%. 순서 문턱→VIP 고정 (순서 뒤바뀔 클레임 1건)
- 수량 -1 입력 시 합계 0원 표시(기대: 오류) / items 없이 제출 시 500 에러
- 쿠폰·배송비: 언급 없음 / “50,000원 정확히” 포함 여부 미확인

Discovery 트랙 — 6단계

- Step1. 각 항목을 증거 수준(L0/L1/L2)으로 분류
- Step2. Invariant/Error/AC/Out-of-Scope 추출
- Step3. ID 부여 (INV-n·E-n·AC-n·OOS-n)
- Step4. 테스트 가능한 문장으로 변환
- Step5. AC는 Given-When-Then으로
- Step6. 출력: ID|계약|증거수준|계층|테스트 아이디어

규칙: 근거 없는 동작을 계약ID로 만들지 마라.

확정 계약표를 docs/PRD.md에 고정 (SSOT)

ID	계약 (불변식 / 에러)	증거 수준	계층
INV-1	subtotal(items) == $\Sigma(\text{price} \times \text{qty})$	— (수학적 정의)	Entity
INV-2	amount $\geq$ 50000 $\rightarrow$ round($\times 0.9$) / 미만 그대로(경계 포함)	L1	Entity
INV-3	문턱 할인 후 VIP면 round($\times 0.95$). 순서 문턱 $\rightarrow$ VIP 고정	L1	Entity
INV-4	모든 입력에서 $0 \leq \text{final_total} \leq \text{subtotal}$	L0	Entity
E-1	items is None $\rightarrow$ TypeError	L0	Boundary*
E-2	price 또는 qty가 음수 $\rightarrow$ ValueError	L0	Boundary*

docs/PRD.md = SSOT(단일 진실). 계약표 + AC(G-W-T) + 구현 금지(OOS: 쿠폰·세금·배송비·포인트)를 포함한다. 아직 “구현 완료”라 쓰지 않는다 — 현재는 RED 준비.

04

장바구니 할인 계산기 — Logic·UI 트랙 구현

4장 · 하류(Delivery). 계약을 코드로 — Logic/UI 두 트랙을 TDD로 완성하고 Review Mode로 마무리.

명세 — 그리고 변하지 않는 진실 계약명세

모든 RED 테스트는 이 표에서 도출한다. 표에 없는 동작은 구현하지 않는다(과잉 구현 금지).

명세 ID	범주	명세(단언)	예시(입력 → 기대)	ECB
INV-1	불변식	최종 금액은 음수가 아니다	<code>final_total(...) ≥ 0</code>	Control
INV-2	불변식	할인 후 금액이 할인 전을 넘지 않는다	<code>discount(a) ≤ a</code>	Control
INV-3	불변식	금액 반환값은 원 단위 정수(반올림)	<code>round → int</code>	Control
E-1	예외·경계	빈 장바구니의 합계는 0	<code>[] → 0</code>	Control·Entity
E-2	예외·경계	문턱 미만은 할인 없음 (< 50,000)	<code>49999 → 49999</code>	Control
E-3	예외·경계	문턱 이상은 10% 할인 (≥ 50,000, 경계 포함)	<code>50000 → 45000</code>	Control
AC-1	인수 기준	합계 = $\Sigma(\text{price} \times \text{qty})$	<code>[1000×3, 2000×2] → 7000</code>	Control·Entity
AC-2	인수 기준	문턱 할인 10% (반올림)	<code>60000 → 54000</code>	Control
AC-3	인수 기준	비VIP 최종 = 문턱 할인 결과	<code>[60000×1], vip=F → 54000</code>	Control
U-1	사용 시나리오	VIP는 문턱 할인 후 5% 추가 할인	<code>[60000×1], vip=T → 51300</code>	Control
U-2	사용 시나리오	주문 품이 VIP 최종 금액을 화면에 표시	<code>GET /?vip=1 → "51,300"</code>	Boundary

① 맘 테스트 — 네 개의 렌즈로 명세 뽑기: ① 항상 참이어야 하는 것 → INV(불변식) ② 경계·잘못된 입력 → E(예외·경계) ③ 정상 흐름의 합격 조건 → AC(인수 기준) ④ 사용자는 실제로 어떻게 쓰나 → U(사용 시나리오).

테스트 플랜 — 요구사항(계약)에서 TC 뽑기

요구사항(계약)을 곧바로 코드로 옮기지 않고, 먼저 “무엇을 어떻게 검증할지”를 TC로 분해한다. 각 행 = RED 테스트 1개.

TC	검증 의도	입력	기대	경계·비고
TC-1	price×qty 합계 반환	[1000×3, 2000×2]	7000	기본 케이스
TC-2	50,000 이상 10% 할인	50000	45000	경계값 ≥50000
TC-3	50,000 미만 그대로	49999	49999	경계값, 미적용
TC-4	문턱할인 후 VIP 5%	[60000×1], VIP	51300	60000→54000→51300

경계값의 힘 — R2를 TC-2·TC-3 두 개로 나눠, 혼한 “초과(>) vs 이상(≥)” 실수를 RED 단계에서 미리 잡는다.

한 사이클 돌려보기 — subtotal (Python)

1

프롬프트

“테스트만” 작성하게 지시 (src/ 금지)

2

실패 로그

ImportError: cannot import 'subtotal'

3

최소 구현

src/cart.py 에 딱 필요한 함수만

4

통과 확인

1 passed → test·feat 커밋 분리

 RED · test_cart.py

```
def test_subtotal_sums_price_times_qty():
    items = [
        {"price":1000,"qty":3},
        {"price":2000,"qty":2},
    ]
    assert subtotal(items) == 7000
```

 GREEN · src/cart.py

```
def subtotal(items):
    return sum(
        it["price"] * it["qty"]
        for it in items)
```

UI 트랙 — 같은 리듬을 화면에 (Flask)

TC	검증 의도	기대
TC-U1	GET "/" 주문 폼 렌더링	200 · qty·vip 포함
TC-U2	GET "/?vip=1" 금액 표시	"51,300" 포함

재사용 — 계산 로직은 새로 만들지 않는다

Logic 트랙의 `final_total`을 그대로 쓴다. "51,300"은 Logic TC-4와 같은 결과 — 두 트랙이 같은 계산을 공유함을 확인하는 연결 고리다.

두 겹의 검증

```
# src/app.py (요약)
@app.route('/')
def index():
    is_vip = request.args.get('vip') == '1'
    total = final_total(ITEMS, is_vip=is_vip)
    return render_template_string(
        TEMPLATE, total=f'{total:,}')

# test_client() → 응답 HTML 빠른 검사
# Simple Browser / 브라우저 MCP → 시각 검증
```

커밋 분리 · Review Mode

git log --oneline

```
feat(ui): 주문 폼 최소 구현 [GREEN]
test(ui): 폼/VIP 실패 테스트 [RED]
feat(logic): final_total [GREEN]
test(logic): final_total [RED]
feat(logic): 문턱 할인 [GREEN]
test(logic): 문턱 할인 [RED]
```

코드를 고치지 않고 AGENTS.md 규칙으로만 점검한다(Plan/Review 모드).

Review Mode 점검 항목 (/review)

- | | | |
|-------------------------------------|--------|---------------------------------|
| ☒ | 우회 | assert True · skip · 예외 삼키기 없는가 |
| ☒ | 과잉 구현 | 요구사항에 없는 코드가 없는가 |
| ☒ | 커밋 분리 | RED와 GREEN이 한 커밋에 안 섞였나 |
| ☒ | 테스트 누락 | 경계값·예외 케이스가 빠지지 않았나 |

05

Gilded Rose — 레거시 코드 QA 프롬프팅

Practical Exercise. Codex CLI + VS Code QA 프롬프트 라이브러리. 0단계(준비) + 1~9단계(QA) + 보너스. Python 3 + pytest + pytest-cov.

실습 개요 & 아이템 규칙 (Python · Codex CLI)

 실습 개요

- 프로젝트: 기존 Gilded Rose 저장소 사용(폴더 복사)
- 브랜치: main에서 prompting 브랜치 신규 생성
- 기술 스택: Python 3 + pytest + pytest-cov
- 규칙 파일: AGENTS.md (프로젝트 루트, 자동 로딩)
- git switch -c feature/qa-prompting # QA 프롬프팅 작업 브랜치

절대 규칙 (AGENTS.md)

- Item 클래스는 절대 수정 금지
- quality 범위 0~50 (Sulfuras 제외)
- sellIn 음수 허용(기한 초과 상태)
- 테스트 Green 상태에서만 리팩토링 진행

일반 아이템	매일 quality -1 기한 후 -2 최솟값 0
Aged Brie	오래될수록 quality ↑ 기한 후 +2 최댓값 50
Backstage Pass	sellIn<11 → +2, <6 → +3 기한 후 quality=0
Sulfuras	quality 항상 80 고정 sellIn 변화 없음
Conjured ★	매일 -2 기한 후 -4 (일반의 2배) ← 신규 기능

9단계 QA 프로세스 개요 (+0단계·보너스)

단계	제목	주요 산출물
0	실습 준비 (브랜치·환경·AGENTS.md)	브랜치 · AGENTS.md
1	요구사항 분석 (Requirements Analysis)	requirements_analysis.md
2	코드 품질 분석 (SOLID & Code Smell)	code_quality_report.md
3	테스트 계획 (Test Plan)	test_plan.md
4	테스트 케이스 작성 (Test Cases)	테스트 코드 파일
5	테스트 실행 & 결함 분석	defect_list.md
6	Golden Master 자동화 (회귀 안전장치)	Golden Master 테스트
7	리팩토링 지원 (Green 유지)	단계별 체크리스트
8	결함 관리 & 보고	defect_report.md
9	QA 종합 검토 (최종 보고서)	qa_final_report.md
보너스	Conjured 기능 추가 통합 구현	Conjured 구현 + Green

핵심 프롬프트 구조 P-C-T-F (Python · Codex CLI)

모든 프롬프트에 Persona · Context · Task · Format 4요소와 공통 제약(Item 수정 금지·경계값 포함)을 넣는다.



Persona

시니어 Python QA / 아키텍트 관점



Context

@gilded_rose.py · Python 3, pytest,
pytest-cov



Task

단일 작업 — 분석·테스트·리팩토링 중
하나



Format

Markdown 표 / 저장 파일명 명시

@gilded_rose.py

[P] 시니어 Python 아키텍트 + 클린코드 전문가입니다.

[T] update_quality() 를 SOLID/Code Smell 관점에서 분석해줘.

- SRP/OCF 위반 지점과 근거
- Code Smell: Long Method, Magic Number, Duplicated Code
- Python 특화: 타입 힌트 부재, 문자열 하드코딩, isinstance 미활용
- 리팩토링 우선순위(1~5)와 이유

[F] Markdown 표. code_quality_report.md 로 저장

결함 심각도 분류 (Defect Severity Matrix)

심각도	Python 예시	설명
Critical	Sulfuras quality 변경됨	즉시 수정 필요. 핵심 비즈니스 로직 파괴
Major	Backstage Pass 구간 경계 오류	기능 오동작. 다음 Sprint 내 수정
Minor	sellIn 0일 처리 순서 오류	부분 오동작. 리팩토링 시 함께 수정
Info	불필요한 import·네이밍 불일치	품질 개선 권고. 다음 리팩토링 시 반영

💡 테스트 실패 시 Codex CLI에 실패 로그(pytest 출력)를 Context로 붙여넣으면 스택트레이스를 읽고 원인·위치(파일:줄)·심각도·최소 수정안을 바로 도출한다.

Conjured 기능 추가 — Item 수정 없이 (Python)

핵심 시험 과제: Item 클래스 수정 없이 Conjured 규칙(매일 -2, 기한 후 -4, 최소 0)을 추가하고 테스트 Green을 확인한다.

Composer 통합 프롬프트

```
@gilded_rose.py @test_gilded_rose.py
@GildedRoseRequirements.txt
```

Conjured 아이템 기능을 추가해줘.

- 규칙: 매일 quality -2,
기한(sell_in < 0) 후 -4, 최소 0
- Item 클래스 수정 없이 구현
- 구현 후 pytest -v 실행해 Green 확인
- pytest --cov 로 커버리지 확인

생성 결과 예시 (Python)

```
def _update_conjured(item):
    rate = 2 if item.sell_in > 0 else 4
    item.quality = max(0, item.quality - rate)

# update_quality() 에서 호출:
# if 'Conjured' in item.name:
#     _update_conjured(item)
```

핵심 — Item 클래스를 수정할 수 없으므로 이름 문자열 기반 분기로 처리한다. 프롬프트에 “Item 클래스 수정 없이” 조건을 반드시 명시한다.



APPENDIX

부록 — 사례 · 이식 · 심화

같은 C2C 구조가 안전(제조 RAG)과 성장(모바일 MVP) 두 도메인에서 어떻게 인스턴스화되는지, 그리고 하네스를 다른 도구로 이식·확장하는 법.

제조 모터 과열 트러블슈팅 RAG (안전 중심)

Mom Test가 드러낸 세 가지 고통

P1 경보 시 여러 PDF를 뒤지느라 초기 대응 10-20분 지연

P2 근거 없는 경험칙 추정에 따른 오진 위험

P3 급박한 상황에서 LOTO 확인이 뒤로 밀리는 안전 절차 누락

ID	계약 (대부분 L0)
INV-2	답변은 반드시 매뉴얼 근거 id/섹션 포함
INV-3	원인/점검/안전/근거 4분할 형식
INV-4	근거 부재 시 '문서상 확인 불가' 강제
E-2	안전 조치(LOTO·보호구) 없는 정비 지시 금지
E-4	실제 설비 제어 기능의 MVP 포함 금지

종단 추적 — 발화 “급하면 LOTO를 나중에 한다” → 고통 P3 → E-2·AC-3 계약화 → 프롬프트 하네스 검사 + 골든 퀘스천 E2E → 하네스 구현 → 안전 문구 포함률 계측으로 귀환. 환각 함정 골든셋(문서에 없어 확신 없던 질문)이 RAG의 거절 유형 원천이 됩니다.

소비자 모바일 그로스 MVP (성장 중심)

정량 목표에서 출발
유입 3,000 · 가입 300 (30일)

job: “30초 안에 나만의 재미있는 결과물을 만들어 SNS에 공유하고 반응을 얻고 싶다”

→ 5개 구현 후보를 ICE로 비교 → AI 스타일 변환 앱 선정

ID	계약 — 성장과 신뢰가 같은 형식으로
INV-1	60초 TTV — 최초 실행 후 60초 내 결과 확인
INV-2	공유 이미지에 앱 설치 딥링크 자동 포함
INV-3	게스트 1회 무료 · 가입 시 워터마크 제거
INV-4	업로드 사진 24시간 내 자동 삭제 (프라이버시)
E-1	변환 실패 시 크레딧을 차감하면 안 된다

AARRR 역산 — 가입 300 → Activation 대비 전환율 약 14.3%, 유입 대비 Activation 70% 가정 → 상위 퍼널 규모 도출. K-factor 0.3으로 바이럴 루프 설계. 성장 계약과 규제 계약(스토어 심사)이 같은 형식으로 공존합니다.

같은 C2C 구조, 도메인별 인스턴스화

구성 요소	사례 1 — 모터 과열 RAG (안전)	사례 2 — 그로스 MVP (성장)
Layer 1 불변식	근거 인용 강제·확인 불가 응답·안전 조치	60초 TTV·공유 딥링크·24h 사진 삭제
Layer 2 UI 계약	역할별 원문 마스킹·4분할 답변	크레딧 소진 시 가입 유도·실패 재시도
Revenue 재정의	비용 절감·안전 리스크 저감	가입 전환
접근 게이트	RBAC 4역할 × 6권한	AccessGate 2상태 × 4기능
품질 지표	groundedness 100%·환각률 0%	TTV ≤ 60초·K-factor ≥ 0.3
테스트 하네스	FakeEmbedder/FakeStore/FakeLLM	Fake Auth/StyleApi/Analytics

도메인 특성은 체계의 구조를 바꾸지 않고 세 지점에서 파라미터로 흡수됩니다 — ① 계약의 내용 ② AARRR의 재정의 ③ 하네스의 Fake 대상. 13단계와 C2C 3요소는 두 도메인에서 누락 없이 동형적으로 유지됩니다.

Claude Code · Codex 협업 — 하네스는 이식된다

2장 하네스(지도 + 설정 + 커맨드 + 금지 + 계약 ID)는 도구 기능이 아니라 “규율”입니다. 파일 이름과 설정 키만 바뀔 뿐, 계약·테스트·금지는 그대로 이식됩니다.

	① Gemini CLI	② Claude Code	③ Codex CLI
지도 파일	GEMINI.md	CLAUDE.md	AGENTS.md
설정 파일	~/.gemini/settings.json	.claude/settings.json	~/.codex/config.toml
계약·테스트·금지	docs/PRD.md · pytest -q — 세 도구 동일		

세 도구가 함께 읽게 하는 법 — Codex 기본 인식 · Gemini는 settings.json의 context.fileName에 추가 · @AGENTS.md 참조 또는 심링크(ln -s AGENTS.md CLAUDE.md). 승인·샌드박스 기본값은 도구마다 다르니 보수적으로 시작(read-only·plan·default).

AI 시대 개발자 역할의 재정의 — 그리고 한계

비교우위의 이동

AI가 계약→구현 비용을 낮춘 상황에서 인간의 비교우위는:

- ① 상류 — 불변식을 발견·계약화한다
- ② 하류 — 계측을 설계해 개념의 실증 여부를 판정한다
- ③ 사슬 전체의 무단절성 감리

→ 코드의 작성자에서 C2C 추적성의 설계자·감리자로 재정의

한계와 타당성 위협

- 통제 실험이 아닌 설계 과학적 시연 · 질적 교차 사례
- 연구자가 설계에 참여한 실행 연구 — 제3자 재현 필요
- AARRR 역산의 전환을 가정은 도메인·시점에 따라 변동
- 향후 — 계약에서 테스트·계측을 자동 생성하는 C2C 도구 체인

본 체계는 새 기법의 발명이 아니라 검증된 기법들의 목적론적 재조직입니다 — 기여는 이들을 C2C라는 단일 목표 아래 배치하고, 그 변환 규칙을 계약 ID와 매트릭스로 형식화한 데 있습니다.

MCP — 외부 도구 연결 (선택 심화)

```
// ~/.codex/config.toml
[mcp_servers.sequential-thinking]
command = "npx"
args = ["-y", "@modelcontextprotocol/server-sequential-thinking"]
[mcp_servers.context7]
command = "npx"
args = ["-y", "@upstash/context7-mcp", "--api-key", "KEY"]
[mcp_servers.github]
url = "https://api.githubcopilot.com/mcp/"
```

무엇이 되나

context7 — 프롬프트 끝에 use context7 → 버전에 맞는 최신 라이브러리 문서를 찾아온다.

github — Fine-grained PAT 발급 후 “내 저장소 목록”, “이슈 생성” 같은 작업.

MCP 서버 역시 하네스의 일부 — 어떤 도구가 연결됐는지 /mcp로 항상 파악하고, 신뢰할 수 없는 서버는 연결하지 않습니다. 변경 후에는 세션을 재시작합니다.

훅(Hooks) — 감사 로그 자동 기록

after_tool_use 훅

```
// ~/.codex/config.toml
[hooks]
[[hooks.after_tool_use]]
match = ".*"
command = ".codex/hooks/audit-log.sh"

# audit-log.sh — 실행마다 한 줄
echo "$(date) | $tool" >> docs/audit.log
```

chmod +x — 실행권한 (필수)

실행권한이 없으면 훅은 조용히 실패합니다.

```
chmod +x .codex/hooks/audit-log.sh
```

ls -l 로 +x 확인.

git log가 “무엇을 커밋했는가”라면, 감사 로그는 “에이전트가 무엇을 실행했는가” — 추적 사슬의 런타임 증거.

훅은 신뢰할 수 있는 스크립트만 등록하고, 로그에 토큰·비밀번호를 남기지 않습니다.



CLOSING

“이 코드 한 줄은 어느 계약을 지키려는 것인가?”

이 질문에 즉시 답할 수 있게 하는 것 — 그것이 C2C입니다. 여러분은 이제 코드의 작성자가 아니라, AI와 협업하며 C2C 추적 사슬을 설계·감리하는 사람입니다.

■ 계약 ID 없는 동작은 구현하지 않는다

■ 실패 로그가 최고의 컨텍스트다

■ 테스트 약화는 즉시 거절한다

■ 확장 뒤에는 수축을 둔다

■ 구조 변경과 동작 변경을 섞지 않는다

■ 매트릭스의 빈칸이 다음 질문이다